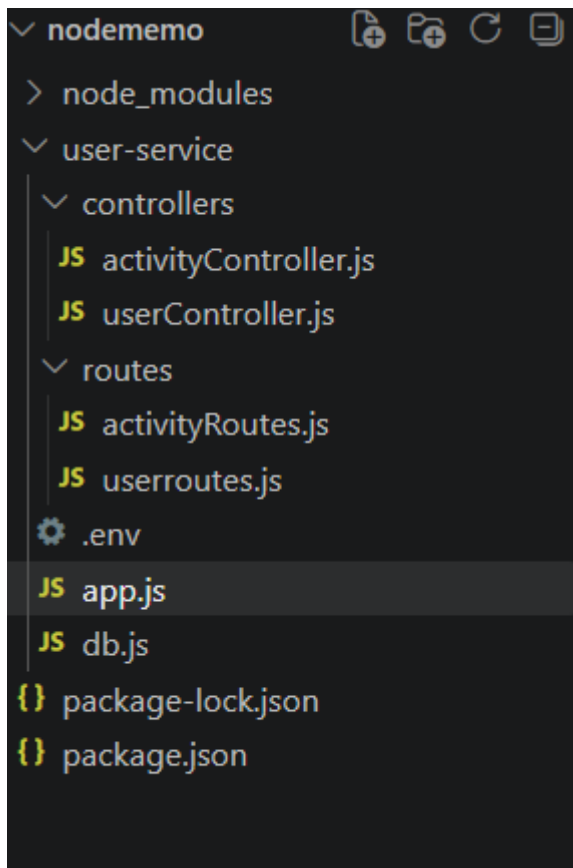


PRACTICAL 7 DBMS



App.js:

```
1  const dns = require("dns");
2  dns.setServers(["8.8.8.8", "8.8.4.4"]);
3
4  require("dotenv").config();
5
6  const express = require("express");
7  const app = express();
8
9  require("./db");
10
11 app.use(express.json());
12
13 const activityRoutes = require("./routes/activityRoutes");
14 const userRoutes = require("./routes/userRoutes");
15
```

```
app.use("/", activityRoutes);
app.use("/", userRoutes);

app.listen(5001, () => {
  console.log("User service running on port 5001");
});
```

Db.js:

```
const mongoose = require("mongoose");

const MONGO_URI = process.env.MONGO_URI;

mongoose.connect(MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => console.log("MongoDB Atlas connected"))
.catch((err) => console.error("MongoDB connection error"));

const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true
  },
  email: {
    type: String,
```

```
14   email: {
15     type: String,
16     required: true
17   },
18   profile: {
19     type: mongoose.Schema.Types.Mixed,
20     default: {}
21   }
22 });
23
24 const activitySchema = new mongoose.Schema({
25   user_id: {
26     type: mongoose.Schema.Types.ObjectId,
27     required: true
```

```
27     type: mongoose.Schema.Types.ObjectId,
28     required: true,
29     ref: "User"
30   },
31   action: {
32     type: String,
33     required: true
34   },
35   timestamp: {
36     type: Date,
37     default: Date.now
38   }
39 });
```

```
38 });
39
40 const User = mongoose.model("User", userSchema);
41 const Activity = mongoose.model("Activity", activitySchema);
42
43 module.exports = { User, Activity };
```

Userroute.js:

```
1  const express = require("express");
2  const router = express.Router();
3
4  const {
5    createUser,
6    getUser
7  } = require("../controllers/userController");
8
9  router.post("/users", createUser);
10 router.get("/users/:id", getUser);
11
12 module.exports = router;
```

Activityroute.js:

```
1  const express = require("express");
2  const router = express.Router();
3
4  const {
5    createActivity,
6    getActivities
7  } = require("../controllers/activityController");
8
9  router.post("/activities", createActivity);
10 router.get("/activities/:user_id", getActivities);
11
12 module.exports = router;
```

Usercontroller.js:

```
1  const { User } = require("../db");
2
3  // POST - Create User
4  const createUser = async (req, res) => {
5    try {
6      const { username, email, profile } = req.body;
7
8      const user = await User.create({
9        username,
10       email,
11       profile
12     });
13
14     res.status(201).json({
15       message: "User created successfully",
```

```
19   } catch (error) {
20     res.status(500).json({
21       message: "Error creating user",
22       error: error.message
23     });
24   }
25 };
26
27 // GET - Get User by ID
28 const getUser = async (req, res) => {
29   try {
30     const user = await User.findById(req.params.id);
31
```

```

38     res.json(user);
39
40   } catch (error) {
41     res.status(500).json({
42       message: "Error getting user",
43       error: error.message
44     });
45   }
46 };
47
48 module.exports = {
49   createUser,
50   getUser
51 };

```

Activitycontroller.js:

```

1  const mongoose = require("mongoose");
2  const { Activity } = require("../db");
3
4  // POST - Create Activity
5  const createActivity = async (req, res) => {
6    try {
7      const { user_id, action } = req.body;
8
9      if (!mongoose.Types.ObjectId.isValid(user_id)) {
10         return res.status(400).json({
11           message: "Invalid user_id. It must be a valid MongoDB
12         });
13       }
14
15       const activity = await Activity.create({

```

```

17     action
18   });
19
20   res.status(201).json({
21     message: "Activity created successfully",
22     activity
23   });
24
25   } catch (error) {
26     res.status(500).json({
27       message: "Error creating activity",
28       error: error.message

```

```

25   } catch (error) {
26     res.status(500).json({
27       message: "Error creating activity",
28       error: error.message
29     });
30   }
31 };
32
33 // GET - Get Activities for a user
34 const getActivities = async (req, res) => {
35   try {
36     const { user_id } = req.params;

```

```

38     if (!mongoose.Types.ObjectId.isValid(user_id)) {
39         return res.status(400).json({
40             message: "Invalid user_id. It must be a valid MongoDB
41         });
42     }
43
44     const activities = await Activity.find({
45         user_id
46     }).sort({ timestamp: -1 });
47
48     res.json(activities);
49
50 } catch (error) {

```

```

51     res.status(500).json({
52         message: "Error getting activities",
53         error: error.message
54     });
55 }
56 };
57
58 module.exports = {
59     createActivity,
60     getActivities
61 };

```

Outputs:

Post:

```

PS C:\Users\koushik\OneDrive\Documents\nodememo> Invoke-WebRequest -Uri "http://localhost:5001/users" -Method POST -Content-Type "application/json" -Body '{"username":"Divya","email":"divya@gmail.com","profile":{}}'
message          user
-----
User created successfully @{username=Divya; email=divya@gmail.com; _id=6ab4ef3ddb100d25078d783b; __v=0}

```

```
message          user
-----
User created successfully @({username=Divya; email=divya@gmail.com; _id=6ab4ef3ddb100d25078d783b; __v=0})

PS C:\Users\koushik\OneDrive\Documents\nodememo> 
```

Get:

```
Pretty-print ☒

{
  "_id": "6ab4ef3ddb100d25078d783b",
  "username": "Divya",
  "email": "divya@gmail.com",
  "__v": 0
}
```

```
[
  {
    "_id": "6ab4f17adb100d25078d783c",
    "user_id": "6ab4ef3ddb100d25078d783b",
    "action": "SEARCH_PYTHON",
    "timestamp": "2026-09-24T09:46:34.473Z",
    "__v": 0
  }
]
```

```
PS C:\Users\koushik\OneDrive\Documents\nodememo> Invoke-RestMethod -Uri "http://localhost:5001/activities" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{"user_id":"6ab4ef3ddb100d25078d783b","action":"SEARCH_PYTHON"}'

message          activity
-----
Activity created successfully @({user_id=6ab4ef3ddb100d25078d783b; action=SEARCH_PYTHON; _id=6ab4f17adb100d25078...
```